

# Gradients & the Geometry of Surfaces

The arrow that points straight uphill

---

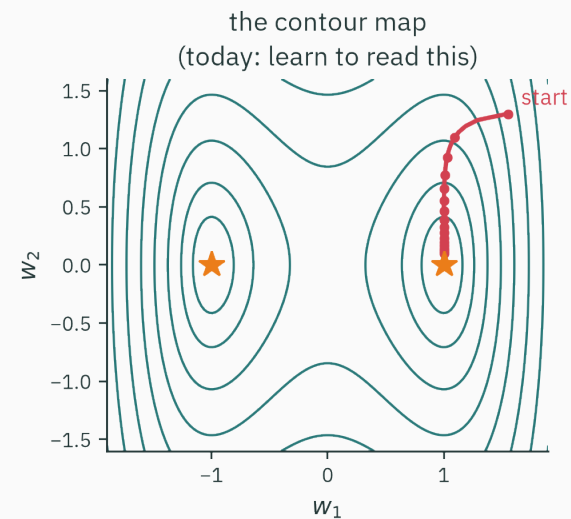
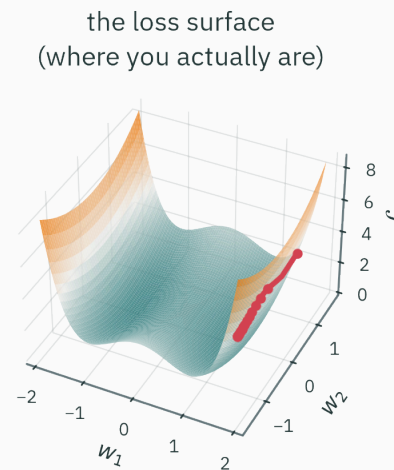
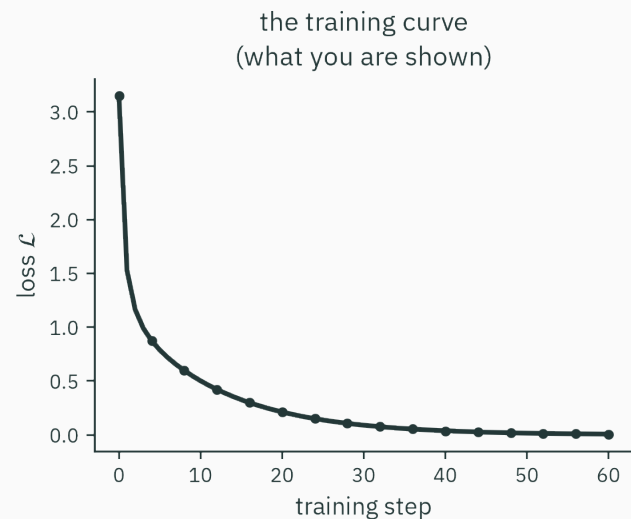
Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

# The gradient of a two-parameter loss

---

# Every training run hides a landscape



You will stare at thousands of the left plot. It is an **altimeter log**: height vs time of a walk. The walk itself happens on the **middle** object — and hikers navigate it with the **right** one.

# Training is a walk you cannot watch

- A real model has **millions** of weights — its loss surface lives in  $\mathbb{R}^{10^6}$ , unplottable forever.
- But every mechanism you'll ever need — slopes, contours, steepest directions — already exists, fully visible, at **two** weights.
- So today is deliberately flat-footed:  $f(x, y)$ , drawn every way we can.

**Training = walking downhill on a loss surface. The training curve is only the altimeter — today we learn to read the **terrain**.**

# Today's one idea

The **gradient** collects every partial sensitivity into one vector — and that vector points **straight uphill**, perpendicular to the contour lines.

Three questions, in order:

1. How do we even **draw** a function of two variables? → surfaces & contour maps
2. How do we **differentiate** it? → partial derivatives, bundled into the gradient
3. Which direction is **steepest**, and why? → the directional derivative ★

# Learning outcomes

By the end of this lecture you will be able to:

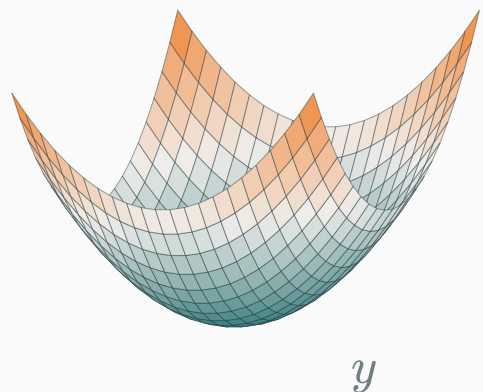
1. Draw one  $f(x, y)$  two ways — **surface** and **contour map** — and translate between them.
2. Read a contour map **like a hiker**: peaks, pits, saddles, steep faces.
3. Compute **partial derivatives** by freezing variables; check them numerically.
4. Assemble the **gradient**  $\nabla f$  and interpret it as an arrow on the map.
5. Predict the slope in **any** direction from just  $\nabla f$  — via  $\nabla f \cdot u$ .
6. Prove ★ that  $\nabla f$  is **perpendicular to contours** and is the **steepest** direction.
7. Run the **multivariate chain rule** along branching pipelines.

# Surfaces and contour maps

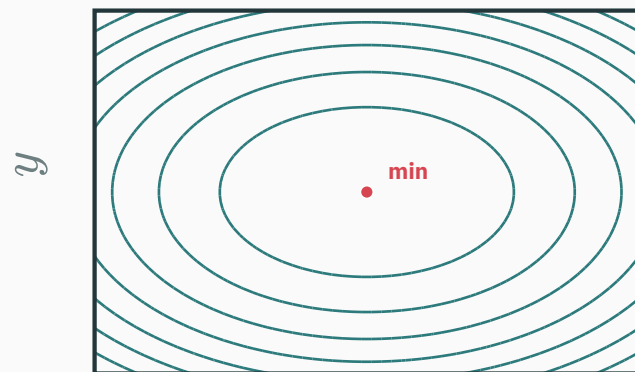
---

# A function of two inputs is a landscape

$f(x, y) = x^2 + 3y^2$  — every map point  $(x, y)$  gets one number: its **height**.



the **surface** view: height drawn as height



the **map** view: height drawn as curves

Same  $f$ , two pictures. The rest of the course lives in the right one — so let's earn it.

# Your first loss landscape, built by hand

Fit the line  $\hat{y} = wx + b$  to two points (1, 2) and (2, 4). The loss is a function of **two** variables — the parameters:

$$\mathcal{L}(w, b) = (2 - (w + b))^2 + (4 - (2w + b))^2$$

| $w$ | $b$ | $\mathcal{L}(w, b)$  |
|-----|-----|----------------------|
| 0   | 0   | $4 + 16 = 20$        |
| 1   | 1   | $0 + 1 = 1$          |
| 2   | 1   | $1 + 1 = 2$          |
| 2   | 0   | $0 + 0 = \mathbf{0}$ |

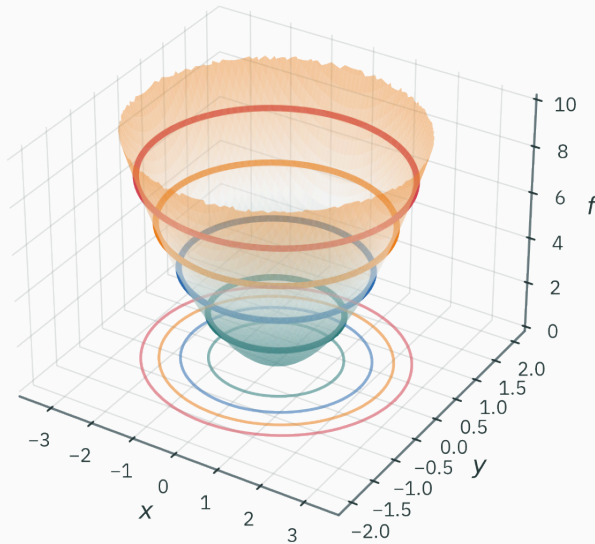
Each row: guess a line, square the two misses, add.

$(w, b) = (2, 0)$  fits perfectly — the data **is**  $y = 2x$ .

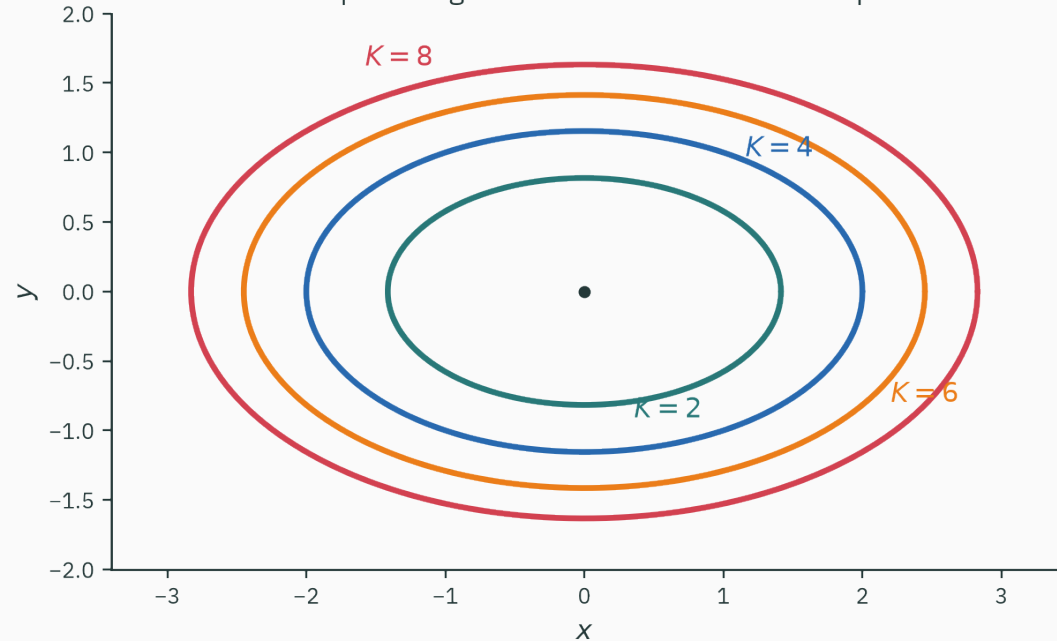
Keep this  $\mathcal{L}$  in mind; it returns at the end of the lecture, with a walker on it.

# How a contour map is made

slice at constant heights  $f(x, y) = K$



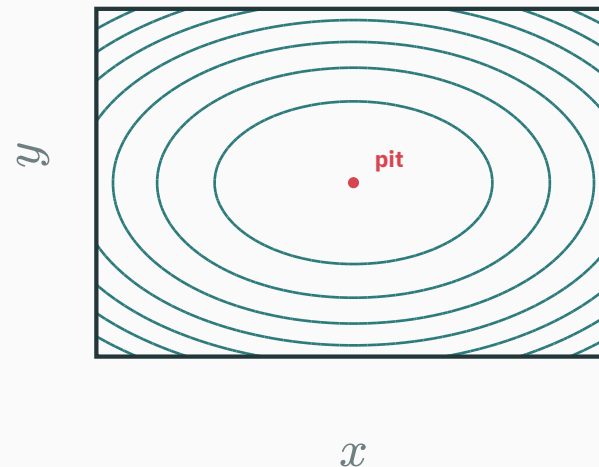
drop the rings on the floor: the contour map



Slice the surface at constant heights  $f(x, y) = K$ ; drop each cut edge straight down.  
A **contour line** (or **level set**) is everything at one height.

# Read the map like a hiker

- **Closed rings shrinking to a dot** — a peak or a pit lives inside.
- **Lines crowded together** — steep face: height changes fast per step.
- **Lines far apart** — gentle plain: many steps to change height.
- **Walking along a line** — you stay exactly level, by definition.



Equal height **spacing** between lines  $\Rightarrow$  line **crowding** = steepness. A contour map is a slope chart in disguise.

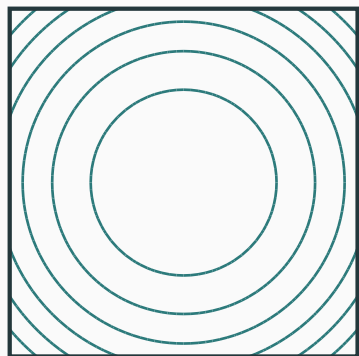
# The map in code · meshgrid + contour

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(-3, 3, 100)
y = np.linspace(-2, 2, 100)
X, Y = np.meshgrid(x, y)      # X[i,j], Y[i,j]: every grid pair
Z = X**2 + 3*Y**2             # f on the whole grid at once
plt.contour(X, Y, Z, levels=10)
plt.gca().set_aspect("equal")
```

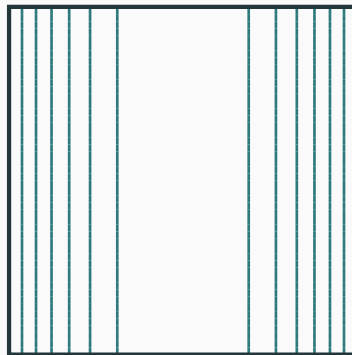
meshgrid tabulates the plane; contour interpolates the level curves between grid values. Every loss-landscape picture you have ever seen is these five lines.

T4 walks through meshgrid slowly — including the classic confusion about which axis is which.

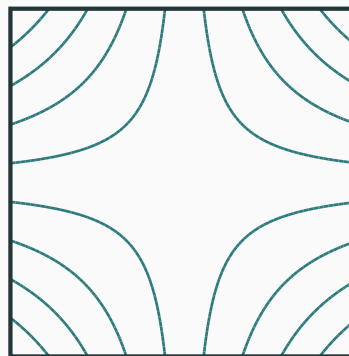
$$x^2 + y^2 \cdot \text{bowl}$$



$$x^2 \cdot \text{trench}$$

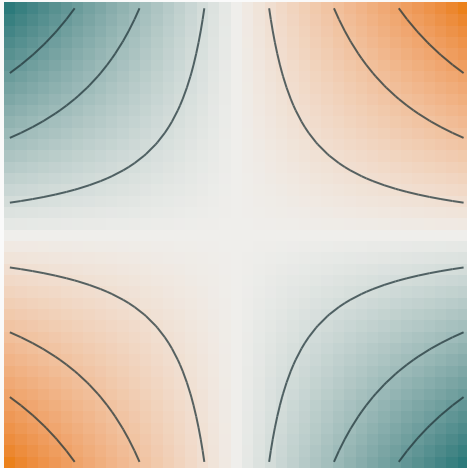


$$xy \cdot \text{saddle}$$



- **Bowl**: rings around one pit — the ideal loss. **Trench**:  $f$  **ignores**  $y$  — a loss that ignores a parameter (a dead weight: contours parallel to its axis).
- **Saddle**: downhill in two quadrants, uphill in two — no ring closes.

# The saddle deserves a second look



heat view: high / low

At the center: flat — yet **neither** a peak nor a pit. Ride  $x = y$  and you climb; ride  $x = -y$  and you sink.

High-dimensional loss surfaces are **full** of saddle points — flat spots that stall training without being minima. L9 gives you the tool (the Hessian) that tells bowls, saddles, and trenches apart.

# Level sets you have already met

Contours are not just a plotting trick — they are how ML **draws decisions**:

- A classifier outputs a probability surface  $p(x, y)$  over the plane; its **decision boundary** is precisely the level set  $p = 0.5$ .
- Every “decision boundary” plot in ES 335 is `plt.contour(X, Y, P, levels=[0.5])` — the same five lines of code as before.

Same object, two costumes: in **parameter space**, level sets of the loss (today); in **input space**, level sets of the model’s output (your next ML course).

# **Partial derivatives: freeze one variable**

---

# A partial derivative varies one coordinate at a time

L7's derivative answers: nudge **the** input, how hard does the output move? Now there are **two** inputs to nudge — and they can move together, or alone.

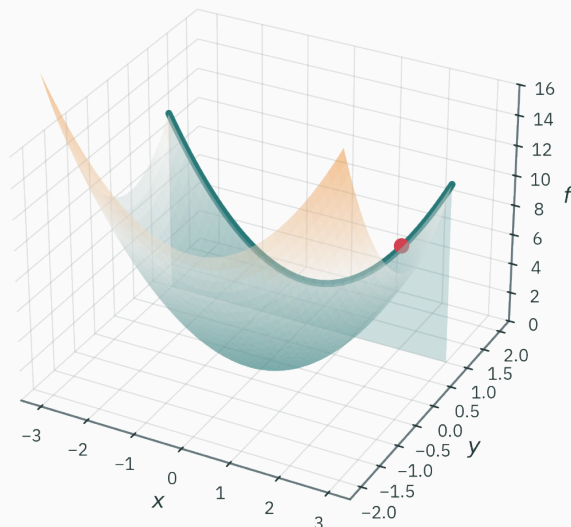
The multivariable move (you will use it for the rest of your life):

**Freeze every variable except one. What remains is a one-variable function — and L7 already taught you everything about those.**

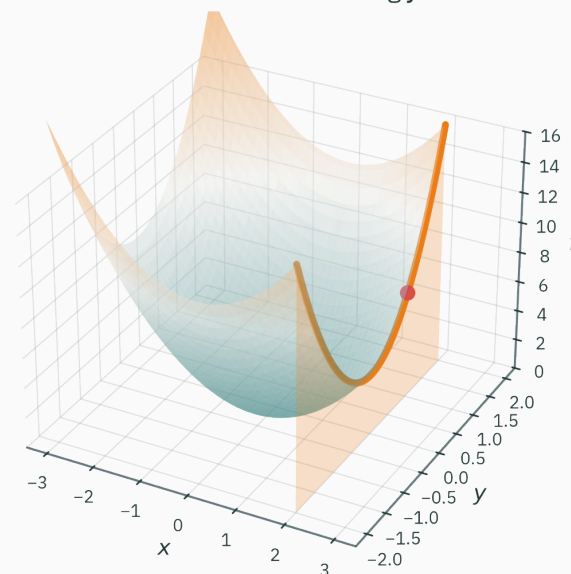
Freezing has a picture, a number, and a symbol. Picture first.

# Freezing a variable sees the surface open

freeze  $y = 1$ : slide along  $x$



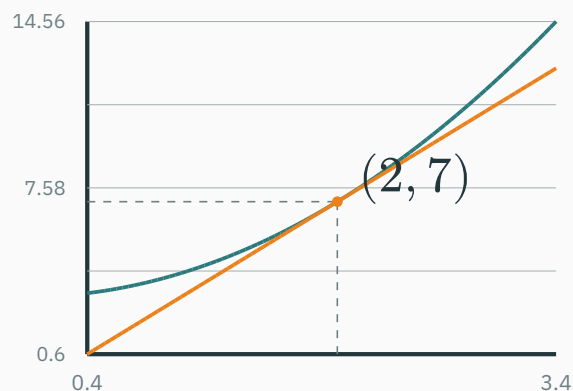
freeze  $x = 2$ : slide along  $y$



Each freeze exposes a **cut edge** through our base point  $(2, 1)$ , where  $f = 7$  (the dot).

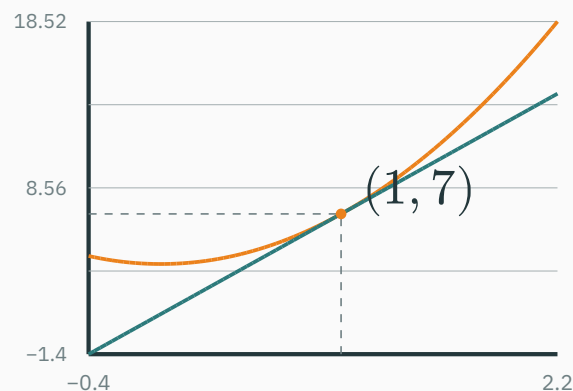
# Each cut edge is an ordinary curve

$$g(x) = f(x, 1) = x^2 + 3$$



$x$

$$h(y) = f(2, y) = 4 + 3y^2$$



$y$

Each cut is a parabola with a **tangent line** at the base point — L7's local-line story, verbatim. Two cuts, two slopes. Let's measure them.

# The nudge table, twice

Base point  $(2, 1)$ ,  $f = 7$ . Nudge **one** coordinate, hold the other — L7's ritual:

| move                      | new value                | response | ratio |
|---------------------------|--------------------------|----------|-------|
| $x : 2 \rightarrow 2.01$  | $f(2.01, 1) = 7.0401$    | 0.0401   | 4.01  |
| $x : 2 \rightarrow 2.001$ | $f(2.001, 1) = 7.004001$ | 0.004001 | 4.001 |
| $y : 1 \rightarrow 1.01$  | $f(2, 1.01) = 7.0603$    | 0.0603   | 6.03  |
| $y : 1 \rightarrow 1.001$ | $f(2, 1.001) = 7.006003$ | 0.006003 | 6.003 |

Two stabilizing ratios: the  $x$ -ratio heads to 4, the  $y$ -ratio to 6.

**At  $(2, 1)$  this surface is 4-steep along  $x$  and 6-steep along  $y$ . Two sensitivities, one point.**

## $\partial$ joins the notation zoo

L7 promised “ $\partial$  joins the zoo in L8”. Here it is — the **partial derivative**:

$$\frac{\partial f}{\partial x}(x_0, y_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h, y_0) - f(x_0, y_0)}{h}$$

- Curly  $\partial$  = “I nudged **one** input; every other input is frozen.”
- To compute: **differentiate as usual, treating the frozen variables as constants**:

$$f = x^2 + 3y^2 : \quad \frac{\partial f}{\partial x} = 2x + 0 = 2x, \quad \frac{\partial f}{\partial y} = 0 + 6y = 6y$$

At  $(2, 1)$ :  $2x = 4 \checkmark$  and  $6y = 6 \checkmark$  — exactly the two stabilized ratios.

# Partials in code

```
import torch
f = lambda x, y: x**2 + 3*y**2
x0, y0 = torch.tensor(2.0), torch.tensor(1.0)

df_dx = torch.func.grad(f, argnums=0)(x0, y0)    # nudge arg 0 only
df_dy = torch.func.grad(f, argnums=1)(x0, y0)    # nudge arg 1 only
print(df_dx, df_dy)
# tensor(4.)  tensor(6.)
```

argnums **is** the freeze: differentiate w.r.t. argument 0, hold the rest. And the values are exact — no 0.0401-style nudge noise. How PyTorch manages that is the whole of L10–L11.

On a loss contour map, a small step in the  $w_2$  direction crosses **three** contour lines; the same-length step in the  $w_1$  direction stays **between** two lines. At that point:

A.  $|\partial \mathcal{L} / \partial w_1| > |\partial \mathcal{L} / \partial w_2|$

B.  $|\partial \mathcal{L} / \partial w_2| > |\partial \mathcal{L} / \partial w_1|$

C. the two partials are equal

D.  $\nabla \mathcal{L} = 0$  there

**Correct: B** — the  $w_2$  partial is bigger

**Why:** Contour lines sit at equal height steps — crossing three of them means the height changed three steps' worth over one small move: a big ratio, i.e. a big  $|\partial\mathcal{L}/\partial w_2|$ . Staying between lines means barely any change: small  $|\partial\mathcal{L}/\partial w_1|$ . D is the opposite of what's described: a zero gradient point has **locally no** crossings in any direction.

**The gradient: every sensitivity  
in one arrow**

---

# Bundle the partials

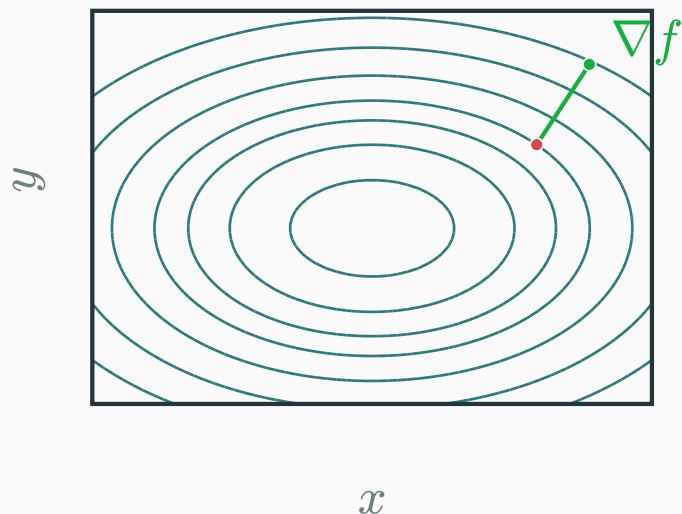
Two sensitivities at every point beg to be one object. Stack them:

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} \quad \text{here: } \nabla f = \begin{pmatrix} 2x \\ 6y \end{pmatrix}, \quad \nabla f(2, 1) = \begin{pmatrix} 4 \\ 6 \end{pmatrix}$$

- $\nabla f$  (“nabla  $f$ ”, or **the gradient**) is a **vector-valued** recipe: feed a point, get a vector.
- It packages **every** first-order fact about  $f$  at that point — L7's derivative, grown up.

Honest bookkeeping: MML writes  $\nabla f$  as a **row** vector (good reasons arrive with the Jacobian). We draw it as a column/arrow today; L9 settles the layout conventions.

# The gradient is an arrow on the map



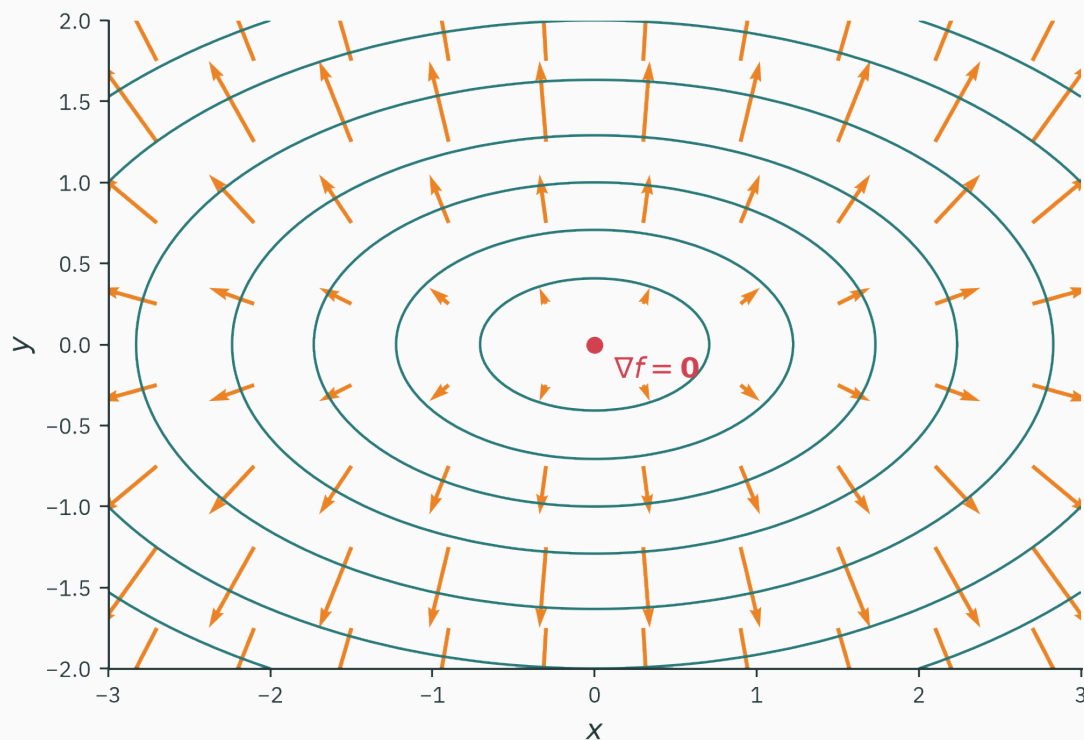
The pin planted at  $(2, 1)$  is  $\nabla f = (4, 6)$ , drawn **in the input plane**.

It leans more along  $y$  than  $x$  ( $6 > 4$ ) — the surface is steeper that way.

The gradient is **not** an arrow on the mountain. It lives on the **map**, pointing along the ground — “walk that way to climb fastest.”

# Drop an arrow at every point: the gradient field

Every arrow is  $\nabla f$  planted at its base point — three regularities hide here:



# What the field is telling us

1. Every arrow points **away from the pit** — uphill, toward higher contours.
2. Every arrow crosses its contour line **at a right angle** — no exceptions, anywhere.
3. Arrows are **long where lines crowd** (steep!), and shrink to  $\nabla f = 0$  at the flat bottom — the resting places of gradient descent (L16–L18's obsession).

**The gradient is perpendicular to the contour through the point, and its norm is the maximum directional derivative.**

# The field in code

```
qx, qy = np.meshgrid(np.linspace(-2.7, 2.7, 10),  
                      np.linspace(-1.7, 1.7, 8))  
U, V = 2*qx, 6*qy          #  $\nabla f = (2x, 6y)$  on the grid  
plt.contour(X, Y, Z, levels=10)  
plt.quiver(qx, qy, U, V)   # arrows over the map
```

quiver = “a case for arrows”. Contours from the **values** of  $f$ , arrows from its **partials** — the two halves of today, overlaid.

**Which way is steepest?**

---

# A hiker's three questions

Standing at  $(2, 1)$  on our surface, gradient in hand —  $\nabla f = (4, 6)$ :

1. If I walk **north-east** — or any direction I fancy — how fast do I climb?
2. Which direction climbs **fastest**?
3. Which directions keep me exactly **level**?

One dot product will answer all three. First, make question 1 precise — with numbers.

# Walking in a chosen direction

Pick a **unit** direction  $u$ , walk along  $(2, 1) + t u$ , and log altitude:  $\varphi(t) = f((2, 1) + t u)$ . The slope  $\varphi'(0)$  is the **directional derivative**  $D_u f$ . Measure it, nudge-style:

| direction $u$                 | measured slope | recognize it?                                                              |
|-------------------------------|----------------|----------------------------------------------------------------------------|
| $(1, 0)$ — due east           | 4              | $\partial f / \partial x$ — of course: freezing $y$ <b>is</b> walking east |
| $(0, 1)$ — due north          | 6              | $\partial f / \partial y$                                                  |
| $(0.6, 0.8)$ — north-east-ish | 7.2            | $0.6 \cdot 4 + 0.8 \cdot 6 = 7.2$ ...a <b>blend</b> of the two             |

The diagonal slope is the partials **weighted by the direction's components**. That pattern is the whole theorem — now we derive it.

## ★ Step 1 · zoom in: the local plane

D DERIVATION

L7: zoom into a smooth **curve** and only a line survives. Zoom into a smooth **surface** and only a **plane** survives — tilted 4-steep along  $x$ , 6-steep along  $y$ :

$$f(2 + h, 1 + k) \approx \underbrace{7}_{f(2,1)} + \underbrace{4}_{\partial f / \partial x} h + \underbrace{6}_{\partial f / \partial y} k$$

Why: move in two legs. East by  $h$ : the  $x$ -cut's tangent charges  $4h$ . Then north by  $k$ : the  $y$ -cut's tangent charges  $6k$ . (To first order the second leg's slope hasn't changed — that's what **smooth** buys.)

**Test it** at  $(h, k) = (0.1, 0.05)$ : plane says  $7 + 0.4 + 0.3 = 7.70$ ; truth  $f(2.1, 1.05) = 7.7175$ . Off by 0.0175 — quadratically small, L7's superlinear story again.

## ★ Step 2 · the slope along $u$ is a dot product

D DERIVATION

Walk the plane in direction  $u = (u_1, u_2)$ : set  $(h, k) = (tu_1, tu_2)$ ,

$$\varphi(t) \approx 7 + t(4u_1 + 6u_2) \Rightarrow \varphi'(0) = 4u_1 + 6u_2$$

That coefficient is exactly a dot product — and nothing about  $(4, 6)$  was special:

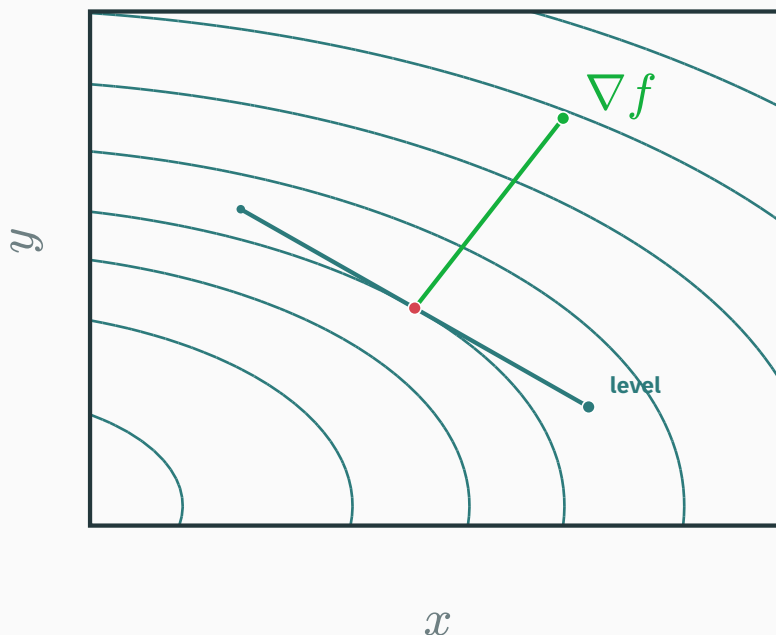
$$D_u f = \nabla f \cdot u$$

Check against the measured table:  $u = (0.6, 0.8)$ :  $(4)(0.6) + (6)(0.8) = 7.2 \checkmark$ . Axes:  
 $(1, 0) \rightarrow 4 \checkmark$ ,  $(0, 1) \rightarrow 6 \checkmark$ .

**One vector,  $\nabla f$ , predicts the slope in **every** direction. That is why the gradient is the only thing an optimizer asks for.**

# The right angle, up close

Zoom onto  $(2, 1)$ : the exact  $\nabla f$  (green pin) vs the level curve  $f = 7$  and its direction (teal chord):



The claim: **always**  $90^\circ$ , at every point of every smooth surface. Picture argument, then algebra.

## ★ Step 3 · level walking $\Rightarrow$ perpendicular

D DERIVATION

The picture argument first. Walking **along a contour**, your altitude is pinned — the level set is the set of one height. No change, no slope:

$D_u f = 0$  whenever  $u$  hugs the contour

Now the algebra finishes it in one line.  $D_u f = \nabla f \cdot u = 0$ , and L3 taught us exactly what a zero dot product **means**:

$\nabla f \cdot u = 0 \Leftrightarrow \nabla f \perp u$ : **the gradient is perpendicular to the level set through its point.**

Numeric witness at  $(2, 1)$ : the contour-hugging direction is  $\propto (3, -2)$ , and  $(4, 6) \cdot (3, -2) = 12 - 12 = 0 \checkmark$ . Claim 2 of the quiver plot — proved.

## ★ Step 4 · steepest ascent, from Cauchy–Schwarz

D DERIVATION

Among all unit  $\mathbf{u}$ , which maximizes  $D_{\mathbf{u}}f$ ? L3's second reading of the dot product:

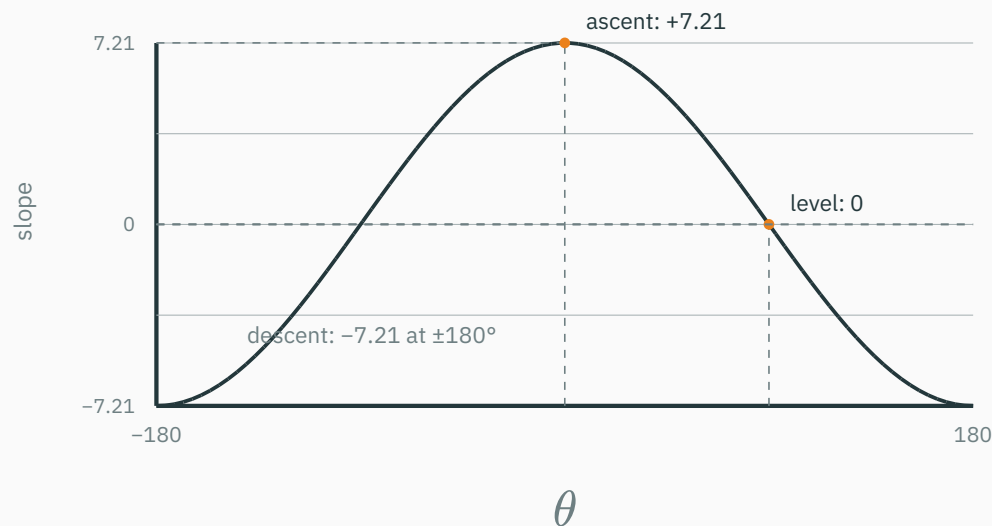
$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \underbrace{\|\mathbf{u}\|}_{=1} \cos \theta = \|\nabla f\| \cos \theta$$

$\cos \theta \leq 1$ , with equality only at  $\theta = 0$  — this is L3's Cauchy–Schwarz, “bought for nothing”:

- **Steepest ascent:**  $\mathbf{u} = \nabla f / \|\nabla f\|$ , slope  $\|\nabla f\| = \sqrt{4^2 + 6^2} = \sqrt{52} \approx 7.21$
- **Steepest descent:**  $-\nabla f$ , slope  $-7.21$  — the direction training will take (L17)

Our hand-picked  $(0.6, 0.8)$  scored 7.2 of a possible 7.21 — it sits just  $3^\circ$  off the true champion  $(4, 6) / \sqrt{52} \approx (0.55, 0.83)$ . Close only counts because  $\cos \theta$  is flat near  $\theta = 0$ .

Slope at  $(2, 1)$  vs the angle  $\theta$  (in degrees) between your step and  $\nabla f$  — a pure cosine:



The directional derivative is maximal along  $\nabla f$ , zero along contour directions, and minimal along  $-\nabla f$ .

**At some point on a surface,  $\nabla f = (3, 4)$ . You step in the unit direction  $u = (4, -3)/5$ . The instantaneous rate of change of  $f$  is:**

**A.** 5

**B.** 0

**C.**  $-5$

**D.** 7

**Correct: B — 0 — you walked along the contour**

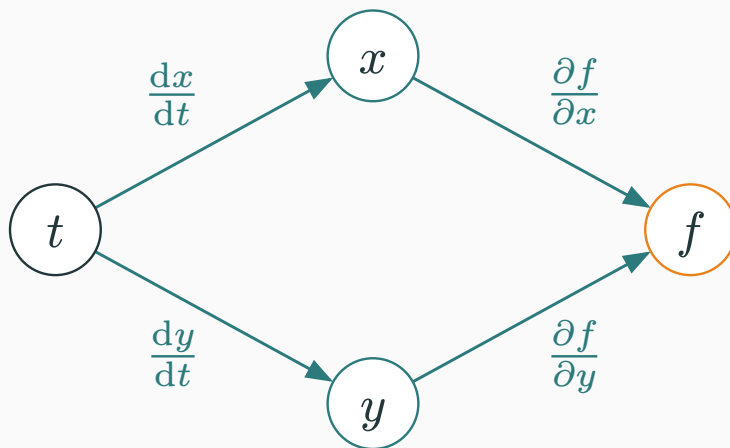
**Why:**  $D_u f = \nabla f \cdot u = (3 \cdot 4 + 4 \cdot (-3))/5 = 0$ : this  $u$  is perpendicular to  $\nabla f$ , so it is a level direction. A is the rate along  $\nabla f / \|\nabla f\|$ ; C is the rate along the negative-gradient direction; D incorrectly adds components.

# **The chain rule grows branches**

---

# Composition, now with two routes

L7's chain rule ran along a single pipeline. In two variables the pipeline **branches** — one clock  $t$  drives both coordinates of a moving point:



$$\frac{df}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

multiply along each route, **add** the routes

# Numbers first: a drone over the bowl

A drone flies the path  $x = t$ ,  $y = t^2$  over our surface  $f = x^2 + 3y^2$ . How fast is the terrain rising under it at  $t = 1$  (position  $(1, 1)$ )?

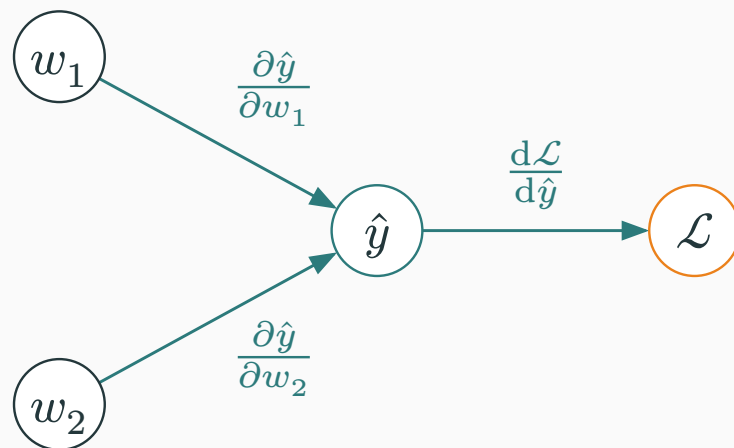
**Route by route.**  $\partial f / \partial x = 2$ ,  $\partial f / \partial y = 6$  at  $(1, 1)$ ;  $dx / dt = 1$ ,  $dy / dt = 2t = 2$ :

$$\frac{df}{dt} = 2 \cdot 1 + 6 \cdot 2 = 14$$

**Sanity check, no chain rule.** Substitute first:  $f(t, t^2) = t^2 + 3t^4$ , so  $df / dt = 2t + 12t^3 = 14$  at  $t = 1 \checkmark$ .

Why **add**? Nudge  $t$  by 0.01: the  $x$ -route delivers  $2 \times 0.01 = 0.02$  of height, the  $y$ -route  $6 \times 0.02 = 0.12$ . Both arrive:  $\Delta f \approx 0.14$ . (Truth:  $f(1.01, 1.0201) = 4.1419$ , i.e.  $\Delta f = 0.1419$ .)

# The chain ML actually runs



- Forward: weights make a prediction, prediction makes a loss — routes **into**  $\mathcal{L}$ .
- Training needs  $\partial \mathcal{L} / \partial w_i$  for **every** weight — the gradient  $\nabla_w \mathcal{L}$ , a million partials.

**Multiply along routes, add the routes — at industrial scale, with every route shared. Doing that bookkeeping without waste is **backpropagation**: L10–L11.**

**Apply the gradient to the  
opening loss**

---

# Training is gradient descent on the landscape

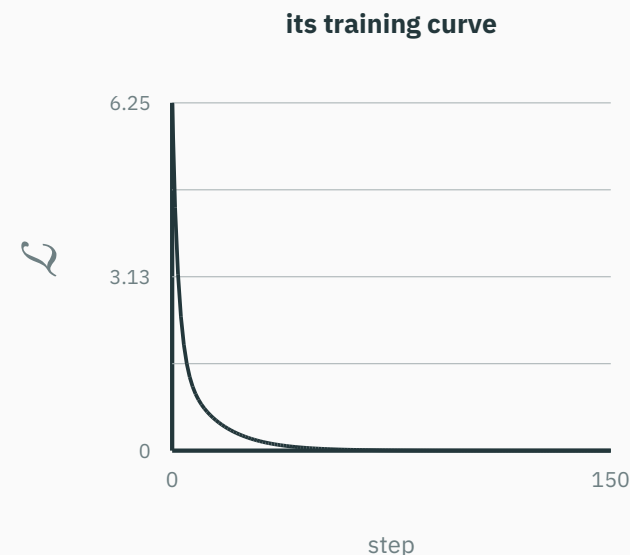
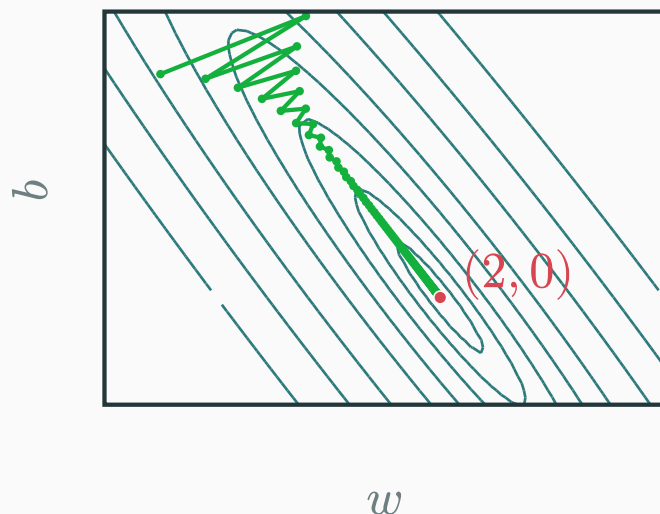
$\nabla \mathcal{L}$  points straight **uphill** — so training repeatedly steps **against** it:

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

- Cross contour lines perpendicularly, downhill, at speed  $\propto$  steepness.
- $\eta$  (the **learning rate**) scales the step; L16–L18 derive its stable range and connect it to curvature.

Watch it run on the loss **you built by hand** — next slide.

Gradient descent on  $\mathcal{L}(w, b)$  from the two-point fit, starting at  $(w, b) = (-0.5, 2.5)$ :



Fast drop, then a long crawl **along the valley floor** toward  $(2, 0)$  — the plateau in every real training curve is usually this: a valley, walked lengthwise.

## INTERACTIVE

↗ [nipunbatra.github.io/interactive-articles/optimizer-race](https://nipunbatra.github.io/interactive-articles/optimizer-race)

Compare gradient descent and other optimizers on 2-D losses. Relate the contour spacing, gradient direction, and trajectory to local curvature.

Try the ill-conditioned valley: watch steps zigzag **across** the valley precisely because each one is perpendicular to its contour.

# Where today's ideas get cashed

| lecture | what it does with today's gradient                                                            |
|---------|-----------------------------------------------------------------------------------------------|
| L9      | derivatives become <b>matrices</b> : Jacobian (many outputs), Hessian (curvature)             |
| L10–L11 | how $\nabla f$ is actually <b>computed</b> : autodiff, backprop, <code>loss.backward()</code> |
| L16–L18 | descent <b>derived and tuned</b> : step sizes, convergence, Newton                            |
| L19–L20 | constrained optima: where the constraint's and the loss's gradients <b>align</b>              |

**Every one of those lectures manipulates the object built today: the vector of all partial sensitivities.**

# Summary & what's next

---

## Lecture 8 — summary

- **Two views**: surface, and contour map of level sets  $f = K$ ; crowding = steepness.
- **Partials**: freeze all but one variable, differentiate as in L7 — ratios 4 and 6 at  $(2, 1)$ .
- **Gradient**:  $\nabla f = (\partial f / \partial x, \partial f / \partial y)$  — one arrow per point, on the **map**.
- **Directional derivative** ★:  $D_u f = \nabla f \cdot u$  — one vector, every slope.
- **Geometry** ★:  $\nabla f \perp$  contours (level walk  $\Rightarrow$  dot = 0); steepest ascent  $\nabla f$  at rate  $\|\nabla f\|$ , steepest descent  $-\nabla f$  — Cauchy–Schwarz, from L3.
- **Chain rule with branches**: multiply along routes, **add** the routes — backprop's skeleton.
- **Training**: each update follows a local descent direction, and the training curve records the objective value along that trajectory.

## Lecture 8 — summary

**Read before L9:** MML §5.2–§5.3 (partial differentiation & gradients). **T4** this week: `meshgrid`, contours, quivers, and your own two-parameter descent loop in NumPy.

# Practice problems

Try on paper; verify in the T4 notebook.

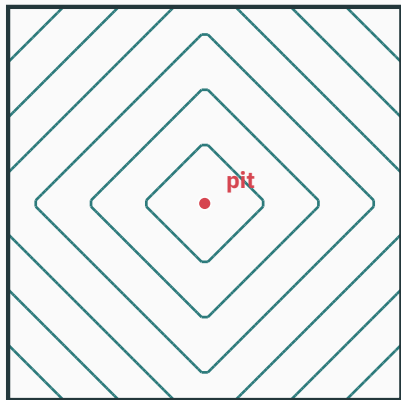
1. For  $f(x, y) = x^3y + y^2$ : compute both partials at  $(1, 2)$  and assemble  $\nabla f$ .
2. Sketch the contour map of  $f(x, y) = 2x + y$ . What shapes are the level sets?  
Compute  $\nabla f$  — why is it the **same arrow everywhere**, and how do the contours confirm it?
3. On our bowl at  $(1, 1)$ : find both unit directions with **zero** instantaneous change, and verify each with a dot product.
4. For  $f = xy$  at  $(2, 3)$ : the slope along  $u = (1, 1)/\sqrt{2}$ , and the steepest slope available there.
5. Let  $x = \cos t$ ,  $y = \sin t$  on the bowl  $f = x^2 + 3y^2$ . Compute  $df/dt$  at  $t = \pi/4$  twice: by the branching chain rule, and by substituting first.
6. A quiver plot shows every arrow pointing **toward** the origin, shrinking near it. Sketch plausible contours, and propose an  $f$  that fits.



## Where the map has corners

★ OPTIONAL

$f(x, y) = |x| + |y|$  — the L1 norm from L3, as terrain:



Diamond rings — with **corners** wherever  $x = 0$  or  $y = 0$ .

At a corner the left and right slices disagree on the slope: **no single tangent, no gradient**. Our smooth-zoom story needs a patch.

At a nondifferentiable corner, a **subgradient** replaces the derivative. This is used for  $L_1$  regularization and ReLU networks; L16 revisits the convex case.

The gradient points where the contour lines crowd.

Next: **Jacobian, Hessian & Multivariate Taylor** — when outputs are vectors too, the derivative becomes a matrix.